Quantifying Customer Dissatisfaction

Complete Project Documentation

DSA Research Project — Final Year | Women's E-Commerce Reviews NLP System

---

Table of Contents

1. Project Overview
2. Architecture Diagram
3. Dataset Description
4. Phase I — Data Engineering
5. Phase II — Hybrid NLP Engine
    - 5.1 VADER Sentiment Analysis
    - 5.2 LDA Topic Modeling
    - 5.3 RoBERTa Sarcasm Detection
6. Phase III — Analytics Dashboard
7. Live Review Analysis Feature
8. How to Run the Project
9. File Structure
10. Technical Dependencies
11. Research Findings & Insights
12. Limitations & Future Work

---

1. Project Overview

Research Question

> *"Can a hybrid NLP system quantify customer dissatisfaction more accurately than conventional positive/negative sentiment classification?"*

Motivation

Traditional sentiment analysis labels reviews as simply "positive" or "negative." This research goes deeper — it **quantifies dissatisfaction on a 0–100 continuous scale**, **identifies which specific** business topic** triggered the dissatisfaction (e.g., Delivery, Fit, Quality), and detects **sarcasm** that misleads rule-based systems. The result is an actionable dissatisfaction intelligence dashboard for e-commerce businesses.

Objectives

#	Objective
1	Build an intelligent data pipeline for multi-source review ingestion
2	Compute a per-review **Dissatisfaction Score (0–100)** using VADER
3	Classify reviews into 6 business topics using **LDA topic modeling**
4	Detect sarcastic/ironic reviews using **RoBERTa transformer model**
5	Visualize all findings in an interactive **Streamlit dashboard**
6	Support **live review testing** with expected vs predicted comparison

---

2. Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────┐
  │                    DATA SOURCES (Phase I)                        │
  │   CSV Dataset  │  Upload CSV/Excel  │  Live Review Text Entry    │
  └──────────────────────────┬──────────────────────────────────────┘
                             │
                     ┌───────▼────────┐
                     │  PREPROCESSING  │  src/preprocessing.py
                     │  - HTML strip   │
                     │  - Slang expand │
                     │  - Lemmatize    │
                     │  - Feature eng  │
                     └────────┬────────┘
                              │
           ┌──────────────────┼──────────────────┐
           │                  │                  │
    ┌──────▼──────┐   ┌───────▼───────┐   ┌──────▼──────────┐
    │    VADER    │   │  LDA TOPICS   │   │  RoBERTa/Irony  │
    │ Sentiment   │   │  Topic Model  │   │  Sarcasm Detect │
    │ + Custom    │   │  6 Categories │   │  Transformer    │
    │  Lexicon    │   │  Gensim LDA   │   │  cardiffnlp     │
    └──────┬──────┘   └───────┬───────┘   └──────┬──────────┘
           │                  │                  │
           └──────────────────┼──────────────────┘
                              │
                     ┌───────▼────────┐
                     │   STREAMLIT    │  app.py
                     │   DASHBOARD    │  7 pages
                     │   (Phase III)  │  Plotly charts
                     └────────────────┘
```

3. Dataset Description

Attribute	Value
Name	Women's Clothing E-Commerce Reviews
Source	Kaggle (public dataset)
Location	`dataset/Womens Clothing E-Commerce Reviews.csv`
Total Records	23,486 reviews
Time Period	Multi-year customer review data

Key Columns

Column	Type	Description
`Review Text`	String	Raw customer review text (primary analysis column)
`Rating`	Integer (1–5)	Star rating given by customer
`Department Name`	String	Product department: Tops, Dresses, Bottoms, Intimate, Jackets, Trend
`Age`	Integer	Reviewer's age
`Recommended IND`	Binary (0/1)	Whether customer recommends the product
`Positive Feedback Count`	Integer	Number of helpful votes received
`Division Name`	String	Division category
`Class Name`	String	Product class

Rating Distribution

Rating	Count	Percentage
⭐ 1	846	3.6%
⭐⭐ 2	1,574	6.7%
⭐⭐⭐ 3	2,866	12.2%
⭐⭐⭐⭐ 4	5,077	21.6%
⭐⭐⭐⭐⭐ 5	13,123	55.9%

4. Phase I — Data Engineering

File: `src/preprocessing.py`

4.1 Why Preprocessing Matters

Raw customer reviews contain noise: HTML tags, URLs, abbreviations, emojis, spelling errors, and e-commerce slang. Without cleaning, all downstream NLP models produce inaccurate results.

4.2 Pipeline Steps

Step 1 — HTML & URL Removal

```
text = re.sub(r'<.*?>', ' ', text)       # strip HTML tags
  text = re.sub(r'http\S+', '', text)      # remove URLs
```

Step 2 — Contraction Expansion

Common contractions are expanded to preserve VADER's negation detection:

- `can't` → `cannot`
- `won't` → `will not`
- `doesn't` → `does not`

Why: VADER uses negation words (`not`, `never`) to flip sentiment polarity. If contractions aren't expanded, VADER misses them.

Step 3 — E-Commerce Slang Normalization

30+ domain-specific abbreviations are expanded:

Slang	Expanded
`tts`	true to size
`ngl`	not going to lie
`tbh`	to be honest
`smh`	shaking my head
`bc`	because
`sz`	size
`gorge`	gorgeous
`fab`	fabulous

Step 4 — Lemmatization (POS-Aware)

Uses **WordNet Lemmatizer** with Penn Treebank POS tags for contextually correct lemmatization:

- `running` (verb) → `run`
- `better` (adjective) → `good`
- Negation words (`no`, `not`, `never`, `n't`) are **preserved** — critical for sentiment accuracy

Step 5 — Feature Engineering

New computed columns added to each review row:

Feature	Formula	Purpose
`word_count`	`len(tokens)`	Review length proxy
`review_length`	`len(clean_text)`	Character count
`exclamation_ratio`	`count('!') / word_count`	Emotional intensity signal
`uppercase_ratio`	`uppercase_chars / total_chars`	Shouting/anger signal
`avg_word_length`	`sum(len(w)) / word_count`	Vocabulary sophistication

4.3 Multi-Source Data Input

The system supports three input modes:

1. **Built-in Dataset** — loads the 23,486-row CSV directly
2. **Upload CSV/Excel** — accepts `.csv`, `.xlsx`, `.xls` with a `Review Text` column
3. **Live Review Text** — instant analysis of manually typed reviews (see Section 7)

---

5. Phase II — Hybrid NLP Engine

The core innovation is combining three complementary NLP approaches, each capturing what the others miss.

5.1 VADER Sentiment Analysis

File: `src/sentiment_vader.py` **Library:** `nltk.sentiment.vader`

What is VADER?

VADER (Valence Aware Dictionary and sEntiment Reasoner) is a lexicon-based rule-set model specifically tuned for social media and short texts. It assigns polarity scores based on a dictionary of ~7,500 sentiment-bearing words.

The Dissatisfaction Score Formula

The core mathematical contribution of this research:

```
Dissatisfaction Score = max(0, -compound) × 100
```

Component	Explanation
`compound`	VADER's normalized sentiment score, range: [−1.0, +1.0]
`-compound`	Inverts polarity: negative reviews → positive dissatisfaction value
`max(0, ...)`	Clips satisfied reviews to 0 (no dissatisfaction)
`× 100`	Scales to human-readable 0–100 range

Examples:

Review	Compound	Dissatisfaction Score
"I love this!"	+0.80	`max(0, -0.80) × 100 = 0.0`
"It's okay"	+0.05	`max(0, -0.05) × 100 = 0.0`
"Slightly disappointed"	−0.35	`max(0, 0.35) × 100 = 35.0`
"Absolute waste of money"	−0.90	`max(0, 0.90) × 100 = 90.0`

Severity Classification

Score Range	Severity Class	Meaning
0 – 4.9	**Satisfied**	Happy customer, no issue
5 – 19.9	**Mildly Dissatisfied**	Minor concern
20 – 44.9	**Moderately Dissatisfied**	Clear complaint
45 – 69.9	**Highly Dissatisfied**	Serious problem
70 – 100	**Severely Dissatisfied**	Extreme anger / full complaint

Custom E-Commerce Lexicon (40+ terms)

VADER's default dictionary lacks e-commerce vocabulary. Custom entries added:

Term	Score	Reason
`overpriced`	−3.2	Strong negative signal
`defective`	−3.5	Product failure
`flimsy`	−2.4	Quality complaint
`shrunk`	−2.2	Sizing after wash
`flattering`	+2.8	Fashion positive
`true to size`	+2.0	Sizing praise
`well-made`	+3.0	Quality praise
`exceeded expectations`	+3.5	Strongest positive

VADER Scores Explained

Each review gets four scores:

- **`pos`** — proportion of text with positive sentiment (0–1)
- **`neu`** — proportion with neutral sentiment (0–1)
- **`neg`** — proportion with negative sentiment (0–1)
- **`compound`** — overall polarity, normalized to [−1, +1]

Note: pos + neu + neg = 1.0

5.2 LDA Topic Modeling

File: `src/topic_modeling.py` **Library:** `gensim`

What is LDA?

Latent Dirichlet Allocation is a probabilistic generative model that discovers latent topics within a document collection. It represents:

- Each **document** as a mixture of topics
- Each **topic** as a distribution over words

Why LDA for Dissatisfaction Research?

Knowing *that* a customer is dissatisfied is useful — knowing *what topic* caused the dissatisfaction is actionable. LDA automatically discovers which business areas (Delivery, Quality, Fit, etc.) generate the most complaints.

The 6 Discovered Topics

Topic ID	Label	Key Words
0	📏 Fit & Size	fit, size, runs, tight, loose, petite
1	🏷️ Product Quality	quality, fabric, material, thin, stitching
2	🚚 Delivery & Shipping	delivery, shipping, arrived, package, late
3	🤝 Customer Service	service, return, refund, exchange, support
4	💰 Value for Money	price, expensive, worth, overpriced, money
5	🎨 Style & Design	style, design, beautiful, color, pattern

LDA Training Configuration

```
LdaModel(
      num_topics = 6,
      passes     = 10,        # training iterations
      alpha      = 'auto',    # per-document topic concentration (auto-tuned)
      eta        = 'auto',    # per-topic word concentration (auto-tuned)
      chunksize  = 200,       # documents per training chunk
      random_state = 42,      # reproducibility
  )
```

Corpus Preprocessing for LDA

The LDA pipeline uses a separate tokenization step:

1. Remove punctuation and lowercase
2. Tokenize with NLTK `word_tokenize`
3. Remove domain-specific stop words (e.g., "dress", "top", "bought")
4. Remove tokens shorter than 4 characters
5. Filter extremes: remove words in < 5 documents or > 60% of documents

Model Quality — Coherence Score (C_v)

The model quality is measured with the **C_v coherence metric** (range 0–1):

- `> 0.50` = Good
- `> 0.60` = Very Good
- `> 0.70` = Excellent

The trained model reports this score on the dashboard for academic transparency.

Topic Dissatisfaction Matrix

A pivot table is computed:

```
Avg Dissatisfaction Score
  ──────────────────────────────────────
  Topic                 | ★1    ★2    ★3    ★4    ★5
  ────────────────────────┼──────────────────────────
  📏 Fit & Size          | 68.2  45.1  22.3   8.1   2.0
  🏷️ Product Quality     | 72.5  50.3  25.8  10.2   1.8
  🚚 Delivery            | 65.0  44.5  20.1   7.5   1.5
  ...
```

This matrix reveals which topics are **most painful at each rating level**.

5.3 RoBERTa Sarcasm Detection

File: `src/sarcasm_detector.py` **Model:** `cardiffnlp/twitter-roberta-base-irony` (Hugging Face)

Why Sarcasm Detection?

VADER is a lexicon-based model. It fails on sarcasm:

> *"Oh great, another item that falls apart after one wash. Wonderful quality."*

VADER reads "great" and "wonderful" as **positive** → compound = +0.68 → Satisfied. But the customer is **severely dissatisfied**. RoBERTa fixes this.

How RoBERTa Works

RoBERTa (Robustly Optimized BERT Pretraining Approach) is a transformer-based language model pre-trained on 160GB of text. The `twitter-roberta-base-irony` variant is fine-tuned specifically to detect irony/sarcasm on social-media-style short texts — making it ideal for customer reviews.
Unlike VADER which looks at individual words, RoBERTa processes the **entire sentence context** using self-attention mechanisms, capturing:

- Semantic contradictions ("great" + "falls apart")
- Ironic intensifiers ("oh wonderful", "just perfect")
- Contradictory clause patterns

Detection Pipeline

```
pipeline = pipeline("text-classification",
                    model="cardiffnlp/twitter-roberta-base-irony",
                    return_all_scores=True)
```

Output per review:

- `irony_prob`: Probability [0.0–1.0] that the review contains irony
- `is_sarcastic`: Boolean — `True` if `irony_prob >= 0.55`
- **Threshold 0.55** chosen to balance precision vs recall

VADER vs RoBERTa Disagreement Analysis

The dashboard flags reviews where VADER says "Satisfied" but RoBERTa says "Sarcastic" — these are the most dangerous misclassifications for a business.

---

6. Phase III — Analytics Dashboard

File: `app.py` **Framework:** Streamlit

Dashboard Pages (7 Pages)

Page 1 — 🏠 Home & Overview

- Research motivation and methodology explanation
- Three-phase pipeline diagram
- Dataset statistics table
- Navigation guide

Page 2 — 📂 Data Hub

- **Three input modes:** Built-in dataset / Upload / Live text
- **Live Analysis Mode** with Expected vs Predicted comparison (see Section 7)
- Raw data preview table with column types and missing value counts
- Key dataset metrics (row count, avg rating, missing text count)

Page 3 — 🔧 Phase I – Preprocessing

- Run preprocessing pipeline with one click
- Before/after text comparison for 5 sample reviews
- Feature statistics: word count distribution, exclamation ratio, uppercase ratio
- Rating distribution bar chart
- Slang normalization examples table
- Box plots of engineered numeric features

Page 4 — 📊 Phase II – Sentiment (VADER)

- **Dissatisfaction Gauge** — speedometer chart showing average index
- **Business KPIs:** Overall Dissatisfaction Index, % Dissatisfied, % Severely Dissatisfied
- **Donut chart** — severity class distribution

- **Histogram** — distribution of dissatisfaction scores
- **Scatter plot** — VADER compound vs star rating
- **Department bar chart** — average dissatisfaction by department
- **Review Explorer** — filterable table of individual reviews with scores
- Top 10 most negative reviews display

Page 5 — 📁 Phase II – Topic Modeling (LDA)

- **Coherence score** metric with academic interpretation
- **Topic keyword cards** — top 8 keywords per topic with probabilities
- **Dissatisfaction heatmap** — topic × rating matrix (colour coded)
- **Topic bar chart** — average dissatisfaction per topic
- **Topic pie chart** — share of reviews per topic
- **Labelled review table** — filterable by topic

Page 6 — 🎭 Phase II – Sarcasm (RoBERTa)

- RoBERTa model loading status
- **Sarcasm donut chart** — sarcastic vs non-sarcastic
- **KPIs:** sarcasm rate %, ironic review count
- **Irony probability histogram** — distribution of RoBERTa scores
- **Disagreement explorer** — reviews where VADER and RoBERTa disagree
- Sample sarcastic review display

Page 7 — 📈 Phase III – Analytics Dashboard

- **Unified KPI row** — all key metrics in one view
- **Word clouds** — overall + negative reviews only
- **Department dissatisfaction bar**
- **Age group box plot**
- **Sarcasm summary donut**
- **Export buttons:** Download analysed data CSV, sarcasm CSV, KPI summary CSV

Visualizations Used

Chart Type	Library	Page
Speedometer Gauge	Plotly `go.Indicator`	Sentiment
Donut Pie Chart	Plotly `go.Pie`	Sentiment, Sarcasm, Dashboard
Histogram	Plotly Express	Sentiment, Sarcasm
Scatter Plot	Plotly Express	Sentiment
Horizontal Bar	Plotly `go.Bar`	Topics, Preprocessing
Heatmap	Plotly `go.Heatmap`	Topics
Box Plot	Plotly Express	Dashboard
Word Cloud	WordCloud + Matplotlib	Dashboard

---

7. Live Review Analysis Feature

This feature allows **instant, real-time testing** of the NLP pipeline on any custom text — without loading the full dataset.

How It Works

Step 1 — Input

Three parallel text areas:

Column	Purpose
Reviews (left)	One review per line
Expected Topic (middle)	Your manual classification per review
Expected Sentiment (right)	Your expected sentiment per review

Step 2 — Pipeline Execution

When "🔍 Analyse Live Reviews" is clicked:

1. **VADER Analysis** runs on each review via `batch_analyze_vader()`
   - Computes compound score, pos/neu/neg ratios, dissatisfaction score, severity class
2. **Keyword-based Topic Classification** runs per review:
   - Each topic has an associated keyword list (18–20 terms)
   - The review text is matched against all lists
   - The topic with the highest keyword hit count wins
   - Fallback: `Product Quality` if no keywords match
3. **Sentiment Label Mapping** from VADER compound:

```
compound ≥ +0.50  →  Positive
  compound ≥ +0.10  →  Neutral-Positive (Mixed)
  compound ≥ -0.10  →  Neutral
  compound ≥ -0.50  →  Neutral-Negative (Mixed)
  compound < -0.50  →  Negative
```

Step 3 — Results Display

Summary Table — one row per review showing:

- Predicted Topic vs Your Expected Topic (with ✅ Match / ⚠️ Mismatch)
- Predicted Sentiment vs Your Expected Sentiment (with ✅ Match / ⚠️ Mismatch)
- Dissatisfaction Score /100
- Severity Class

Per-Review Detail Cards (expandable) showing:

- Full review text
- Predicted Topic with expected comparison
- Predicted Sentiment with expected comparison
- Severity class with colour-coded badge
- VADER score breakdown table (Positive / Neutral / Negative / Compound)
- Colour-coded Dissatisfaction Score badge (green→yellow→orange→red)

Live Analysis Example

Input Review: `"The package arrived two weeks late and was completely damaged."`
Expected Topic: `Delivery & Shipping` **Expected Sentiment:** `Negative`

System Output:

Field	Value
Predicted Topic	🚚 Delivery & Shipping ✅ Match
Predicted Sentiment	Negative ✅ Match
Dissatisfaction Score	64.8/100
Severity Class	Highly Dissatisfied
VADER Positive	0.000
VADER Neutral	0.677
VADER Negative	0.323
VADER Compound	−0.6478

8. How to Run the Project

Prerequisites

- Python 3.9+ (tested on Python 3.10)
- Windows / macOS / Linux

Installation

```
# 1. Navigate to project directory
  cd "sentiment-Analyzer-new-main-11"

  # 2. Install all dependencies
  pip install streamlit pandas numpy nltk gensim plotly matplotlib seaborn \
              wordcloud scikit-learn pyLDAvis openpyxl scipy transformers torch

  # 3. Download NLTK data (automatic on first run, or manually):
  python -c "import nltk; nltk.download('vader_lexicon'); nltk.download('punkt'); nltk.
  download('stopwords'); nltk.download('wordnet'); nltk.download('averaged_perceptron_t
  agger')"
```

Running the Dashboard

```
# Windows (PowerShell) — recommended:
  $env:PYTHONUTF8='1'
  python -m streamlit run app.py --server.port 8501

  # macOS / Linux:
  PYTHONUTF8=1 python -m streamlit run app.py --server.port 8501
```

Then open: **http://localhost:8501**

> ***Note:*** `PYTHONUTF8=1` *is required on Windows to handle emoji characters correctly in the terminal. Without it you may see* `UnicodeDecodeError`.

Recommended Workflow (Full Dataset Analysis)

1. Go to 📂 **Data Hub** → Select "Use Built-in Dataset" → Click "Load Dataset"
2. Go to 🔧 **Phase I** → Click "Run Preprocessing Pipeline"
3. Go to 📊 **Phase II – Sentiment** → Click "Run VADER Analysis"
4. Go to 📁 **Phase II – Topics** → Click "Train LDA Model"

5. Go to 🤖 **Phase II – Sarcasm** → Click "Run Sarcasm Detection"
6. Go to 📈 **Phase III – Dashboard** → View unified results + export CSVs

Quick Live Testing (No Dataset Needed)

1. Go to 📂 **Data Hub**
2. Select "✏️ Enter Live Review Text"
3. Type reviews in the left column
4. Optionally enter expected topics/sentiments in middle/right columns
5. Click "🔍 Analyse Live Reviews"
6. Results appear immediately below

---

9. File Structure

```
sentiment-Analyzer-new-main-11/
  │
  ├── app.py                          # Main Streamlit dashboard (7 pages, ~1,000 line
  s)
  ├── requirements.txt                # Python package list
  ├── Project.md                      # Original research brief
  │
  ├── dataset/
  │   └── Womens Clothing E-Commerce Reviews.csv   # 23,486 reviews
  │
  └── src/
      ├── __init__.py                 # Package marker
      ├── preprocessing.py            # Phase I — Data Engineering
      ├── sentiment_vader.py          # Phase II — VADER + Custom Lexicon
      ├── topic_modeling.py           # Phase II — Gensim LDA
      ├── sarcasm_detector.py         # Phase II — RoBERTa Irony Detection
      └── visualizations.py          # Phase III — All Plotly/Matplotlib charts
```

Module Responsibilities

File	Lines	Responsibility
`app.py`	~990	Dashboard UI, page routing, session state, CSS
`src/preprocessing.py`	148	Text cleaning, slang, lemmatization, features
`src/sentiment_vader.py`	108	VADER SIA, dissatisfaction formula, KPIs
`src/topic_modeling.py`	144	LDA corpus, training, topic assignment
`src/sarcasm_detector.py`	~90	RoBERTa pipeline, sarcasm detection
`src/visualizations.py`	266	All 12 chart functions (Plotly + Matplotlib)

---

10. Technical Dependencies

Package	Version	Purpose
`streamlit`	≥1.28	Dashboard framework
`pandas`	≥1.5	Data manipulation
`numpy`	≥1.24	Numerical operations
`nltk`	≥3.8	VADER, tokenization, lemmatization
`gensim`	≥4.3	LDA topic modeling
`plotly`	≥5.15	Interactive charts
`matplotlib`	≥3.7	Word cloud rendering
`wordcloud`	≥1.9	Word cloud generation
`scikit-learn`	≥1.2	Utility ML functions
`transformers`	≥4.30	RoBERTa model (Hugging Face)
`torch`	≥2.0	PyTorch backend for RoBERTa
`scipy`	≥1.10	Statistical functions
`openpyxl`	≥3.1	Excel file reading

---

11. Research Findings & Insights

Key Finding 1 — VADER Dissatisfaction Distribution

The Women's Clothing dataset shows a highly skewed distribution:

- ~56% of reviews are "Satisfied" (5-star dominated)

- Only ~10% fall in "Highly/Severely Dissatisfied" range
- This skew makes the 0–100 continuous score more informative than binary classification

Key Finding 2 — Topic-Level Dissatisfaction

Preliminary analysis shows these average dissatisfaction scores by topic:

- 🏷️ **Product Quality** — highest average dissatisfaction (most common complaint)
- 📏 **Fit & Size** — second highest (sizing inconsistency in fashion)
- 💰 **Value for Money** — strong negative correlation with 1-star reviews

Key Finding 3 — Sarcasm Blindspot

VADER misclassifies approximately 3–8% of reviews due to sarcasm. Reviews like "Oh great, completely see-through and unwearable" receive positive compound scores because of words like "great," "completely," and high word count.

Key Finding 4 — Compound Score vs Star Rating

The scatter plot (Page 4) reveals an interesting finding: **many 3-star reviews have compound scores below −0.3**, suggesting customers give moderate star ratings but express strong dissatisfaction in text. This validates the text-based approach over rating-only analysis.

---

12. Limitations & Future Work

Current Limitations

Limitation	Impact
VADER is lexicon-based	Cannot understand complex negation chains or sarcasm
LDA requires large corpus	Live mode uses keyword matching instead of true LDA
RoBERTa is irony-trained (Twitter)	May not perfectly transfer to product reviews
Single language (English only)	Cannot analyze non-English reviews
No temporal trend analysis	Does not track dissatisfaction changes over time

Future Enhancements

1. **Aspect-Based Sentiment Analysis (ABSA)** — Extract opinion targets at word level (e.g., "the *zipper* is broken" → zipper quality = negative)
2. **Fine-tuned RoBERTa** — Fine-tune `roberta-base` directly on e-commerce review sarcasm instead of Twitter irony
3. **Multi-language Support** — Add mBERT or XLM-R for multilingual dissatisfaction analysis

4. **Temporal Trend Analysis** — Time-series chart showing dissatisfaction index over review dates
5. **Named Entity Recognition (NER)** — Extract specific product names and brands causing dissatisfaction
6. **Real-time API** — REST API wrapper so businesses can send reviews programmatically
7. **Automated Alert System** — Flag when dissatisfaction index exceeds a threshold and notify via email/Slack

---

Appendix A — VADER Compound Score Interpretation

Compound Range	Sentiment	Dissatisfaction Score	Business Meaning
+0.5 to +1.0	Very Positive	0.0	Loyal customer, promoter
+0.1 to +0.5	Positive	0.0	Generally happy
−0.1 to +0.1	Neutral	0–10	Indifferent, lukewarm
−0.5 to −0.1	Negative	10–50	Mildly to moderately upset
−1.0 to −0.5	Very Negative	50–100	Severely dissatisfied

Appendix B — Keyword Topic Classifier (Live Mode)

The live analysis uses keyword matching to classify topics because LDA (trained on 23K reviews) cannot generalize to 1–5 review inputs. Each topic has ~18 keywords:

```
'🚚 Delivery & Shipping': [
      'delivery','shipping','ship','arrived','package','parcel',
      'courier','late','delay','damaged','tracking','lost','dispatch'
  ]
  '📏 Fit & Size': [
      'size','fit','fits','small','large','tight','loose',
      'runs','petite','length','short','long','wide','baggy'
  ]
  # ... (4 more topics)
```

Scoring: Count of keyword hits per topic → assign highest-scoring topic.

Document prepared for DSA Final Year Research Project System: Quantifying Customer Dissatisfaction — Hybrid NLP Approach Tools: Python 3.10 · NLTK · Gensim · Hugging Face Transformers · Streamlit · Plotly